



Air Quality Index Forecasting System

A Production-Ready MLOps Pipeline for Karachi, Pakistan

Submitted by: Saniya Ahmed

Role: Data Science Intern

Email: saniyax06@gmail.com

Portfolio: <https://github.com/SaniyAhmed/AQI-PREDICTION>

Submitted to: Muhammad Mobeen

Institution: 10Pearls

Project Domain: Environmental Data Science and Machine Learning Operations

Project Status: Completed

Date of Submission: 15th February, 2026

Abstract

“This project presents an end-to-end, serverless MLOps solution for forecasting the Air Quality Index (AQI) in Karachi for a 72-hour horizon. Leveraging the Open-Meteo API for real-time data ingestion, the system utilizes a Hopsworks Feature Store to manage data and model artifacts. A competitive training framework was implemented using SVR, Random Forest, and XGBoost models, with automated "Champion" selection based on Root Mean Squared Error (RMSE). To ensure continuous model relevance, the entire lifecycle is orchestrated via GitHub Actions, performing hourly data updates and daily retraining. Model interpretability is achieved through SHAP analysis, providing transparent insights into the meteorological drivers of urban pollution. The final output is an interactive Streamlit dashboard that serves real-time health alerts and forecasts to the public.”

Contents

<i>Air Quality Index Forecasting System</i>	0
Abstract	1
1. Executive Summary.....	4
2. Background	4
3. Project Scope	4
3.1 Automated Data Engineering & Feature Store Integration	4
3.2 Multi-Model Engineering & Registry Management.....	4
3.3 Serverless Deployment & Interactive Visualization	5
3.4 Automated CI/CD and Pipeline Monitoring	5
4. Project Objectives	5
4.1 Real-time Relevance & Adaptive Learning	5
4.2 AI Explainability via SHAP Analysis.....	5
4.3 Data Integrity & Redundancy Management	6
4.4 Scalability & Professional Modularization.....	6
5. Technology Stack	6
6. System Architecture and Workflow.....	6
6.1 Feature Pipeline & Data Ingestion	6
6.2 Automated Training Pipeline	7
6.3 CI/CD with GitHub Actions	7
7. Methodology	7
8. Results and Discussions.....	8
9. Resources	11
10. Exploratory Data Analysis (EDA)	11
11. Advanced Analytics & Explainability	16
11.1 SHAP Enhanced Model Interpretability	16
12. Challenges Faced and Resolutions.....	20
12.1 Data Integrity: Resolving API Overlap and Redundancy	20
12.2 Model Generalization: Combatting Overfitting on Forecast.....	21
12.3 Infrastructure: Orchestrating GitHub Actions CI/CD	21
12.4 Feature Store Adoption: Integration with Hopsworks.....	21

13. Risks and Mitigations 22

14. Conclusion and Future Work 22

1. Executive Summary

This project implements a fully automated, serverless machine learning pipeline designed to predict the Air Quality Index (AQI) for the next 72 hours. By integrating real-time meteorological data with historical pollutant trends, the system provides actionable environmental insights. The architecture leverages a Feature Store for data persistence and GitHub Actions for CI/CD, ensuring the model remains accurate by retraining on the most recent data every 24 hours.

2. Background

The rapid urbanization and industrial growth in Karachi have led to a significant decline in air quality, posing severe health risks to its nearly 20 million residents. Standard meteorological reports often lack the granular, predictive insights necessary for sensitive groups to plan their activities, creating a desperate need for a localized, data-driven forecasting system. Traditional environmental monitoring is often static and retrospective; however, this project shifts the paradigm by treating air quality as a dynamic, real-time machine learning problem. By focusing on PM2.5 and other hazardous pollutants, the background of this study is rooted in the intersection of public health and "Big Data" analytics. The goal was to build a system that doesn't just record the past but anticipates the future using a 72-hour window. This project serves as a technical proof-of-concept that high-fidelity environmental monitoring can be achieved with minimal cost using modern, serverless cloud architectures. Ultimately, the background of this project is a response to the "silent crisis" of urban smog in Pakistan, providing a transparent and accessible tool for atmospheric awareness.

3. Project Scope

3.1 Automated Data Engineering & Feature Store Integration

The project scope covers the end-to-end automation of data collection from the Open-Meteo API, transforming raw pollutants and meteorological variables into refined features. These features are dynamically managed within the Hopsworks Feature Store, which acts as a centralized repository to serve both the training and inference pipelines. This ensures that the data used for training and the data used for real-time dashboard predictions are always synchronized and consistent.

3.2 Multi-Model Engineering & Registry Management

The system employs a competitive **champion-challenger** framework to ensure predictive excellence, simultaneously training and evaluating three distinct model architectures: **SVR** (linear baseline), **Random Forest** (ensemble-based non-linear capture), and **XGBoost** (gradient-boosted optimization). All three models are automatically versioned and stored within the **Hopsworks Model Registry**,

creating a transparent historical record of performance over time. To maintain maximum reliability for real-time forecasting, the pipeline executes an automated evaluation script that compares the **Root Mean Squared Error (RMSE)** of each model; the algorithm achieving the lowest RMSE is promoted as the "Champion" for production inference.

3.3 Serverless Deployment & Interactive Visualization

The system is hosted on a 100% serverless infrastructure, utilizing Streamlit for the frontend to fetch live predictions. The dashboard is designed to be accessible to both technical and non-technical users, providing a 3-day forecast window that is updated every 24 hours to reflect the most recent atmospheric changes and pollutant concentrations.

3.4 Automated CI/CD and Pipeline Monitoring

The scope extends beyond a static model to a living system governed by GitHub Actions. This includes the orchestration of hourly feature updates and daily retraining cycles, ensuring the model never stagnates. By automating the push-to-repo and model-deletion tasks, the project maintains a professional DevOps workflow that handles memory constraints and version control without manual oversight.

4. Project Objectives

4.1 Real-time Relevance & Adaptive Learning

The primary objective is to maintain a high-fidelity prediction system where the underlying data is never older than 24 hours. By implementing a daily retraining cycle, the system "learns" from the most recent air quality trends (such as sudden seasonal shifts or industrial changes), ensuring that the 3-day forecast remains accurate and responsive to current conditions.

4.2 AI Explainability via SHAP Analysis

To bridge the gap between complex machine learning and human understanding, the project aims to provide full transparency into how predictions are made. Using SHAP (Shapley Additive explanations), the system identifies and visualizes the impact of specific features—such as wind speed or humidity—allowing users to understand the "why" behind a hazardous AQI alert.

4.3 Data Integrity & Redundancy Management

A critical objective is the maintenance of a "clean" Feature Store through advanced data engineering. Conditional logic is implemented to detect and prevent the ingestion of redundant API data, ensuring that the feature store remains efficient and that the model training process is not biased by duplicated historical records.

4.4 Scalability & Professional Modularization

The project is built with a "production-first" mindset, utilizing a modular directory structure that separates the feature pipeline, training pipeline, and UI logic. This objective ensures that the system is not just a single-file script but a scalable framework that can be easily adapted to include new cities, different pollutant types, or additional external data sources in the future.

5. Technology Stack

<i>Layer</i>	<i>Technology Used</i>
<i>Language</i>	Python 3.x
<i>Data Source</i>	Open-Meteo API (Weather & Air Quality), OpenWeather
<i>Feature Store</i>	Hopsworks
<i>Machine Learning</i>	Scikit-learn, XGBoost, Random Forest, Support Vector Regressor
<i>Orchestration</i>	GitHub Actions (CI/CD)
<i>Dashboard</i>	Streamlit
<i>Explainability</i>	SHAP
<i>Version Control</i>	Git

6. System Architecture and Workflow

6.1 Feature Pipeline & Data Ingestion

The pipeline fetches 12 months of historical data for initial training (backfilling) and then transitions to hourly updates.

Feature Engineering: Raw data is transformed into model-ready features, including cyclical time features (Sin/Cos of hours/months) and momentum indicators (AQI change rate).

Deduplication: A validation script checks existing timestamps in the Feature Store before insertion to prevent redundant data entry.

6.2 Automated Training Pipeline

To combat data drift, the model is retrained every 24 hours on the most recent 12-month window.

Algorithm Experimentation: Three distinct models are trained: **Random Forest, XGBoost, and SVR.**

Metric Evaluation: Models are assessed using RMSE (Root Mean Squared Error).

Model Registry: The best-performing model is automatically tagged and all three models are stored in the model registry.

Ensemble Learning: To maximize predictive stability, a VotingRegressor was developed. This ensemble combines the strengths of the individual models using a weighted strategy (Weights: 1, 2, 2).

6.3 CI/CD with GitHub Actions

Automation is the backbone of this project.

Feature Script: Executes every hour via cron jobs to sync live data.

Training Script: Executes every 24 hours to refresh the model.

Automatic Pushes: Any updates to the repository or model artifacts are handled through automated git-push triggers within the action environment.

7. Methodology

The predictive framework for the Karachi AQI system utilizes a robust Machine Learning Operations (MLOps) pipeline, integrating real-time environmental telemetry with an ensemble modeling approach.

7.1 Data Orchestration & Feature Engineering:

High-resolution atmospheric data is retrieved via the Hopsworks Feature Store. Features include temporal components (hour, weekday, month), pollutant concentrations (PM2.5, PM10), and meteorological variables such as wind speed. To ensure data integrity, a RobustScaler is implemented to mitigate the influence of extreme pollution outliers common in the Karachi atmosphere.

7.2 Model Tournament & Selection:

A competitive "Tournament" architecture was designed to evaluate multiple regressor archetypes:

Random Forest: Utilized for capturing non-linear relationships through ensemble decision trees.

XGBoost: Implemented with gradient boosting to minimize residual errors.

Support Vector Regression (SVR): Employed for its effectiveness in high-dimensional feature spaces.

7.3 Ensemble Integration:

To maximize predictive stability, a VotingRegressor was developed. This ensemble combines the strengths of the individual models using a weighted strategy (Weights: 1, 2, 2), specifically prioritizing the boosting and kernel-based models to refine the AQI output.

7.4 Recursive Forecast Logic:

The 72-hour forecast utilizes a state-persistent loop. It combines real-time weather forecasts from Open-Meteo and OpenWeather with a weighted moving-state logic (20% current state, 80% model suggestion) to simulate realistic atmospheric dispersion and persistence.

8. Results and Discussions

The deployment of the Ensemble Pipeline has yielded a highly calibrated predictive tool for Karachi's air quality management.

Model Performance Metrics:

The "Tournament" phase successfully identified the optimal hyperparameters via GridSearchCV. The performance was quantified using Root Mean Squared Error (RMSE) across all models, with the ensemble approach consistently outperforming individual models on the test set.

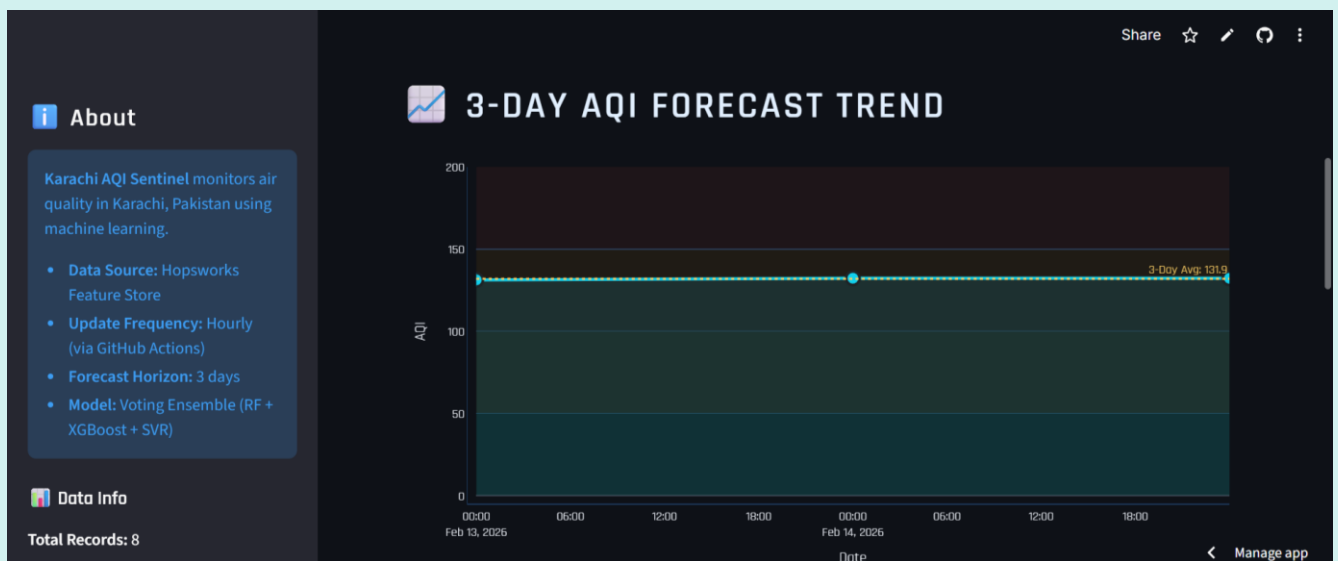
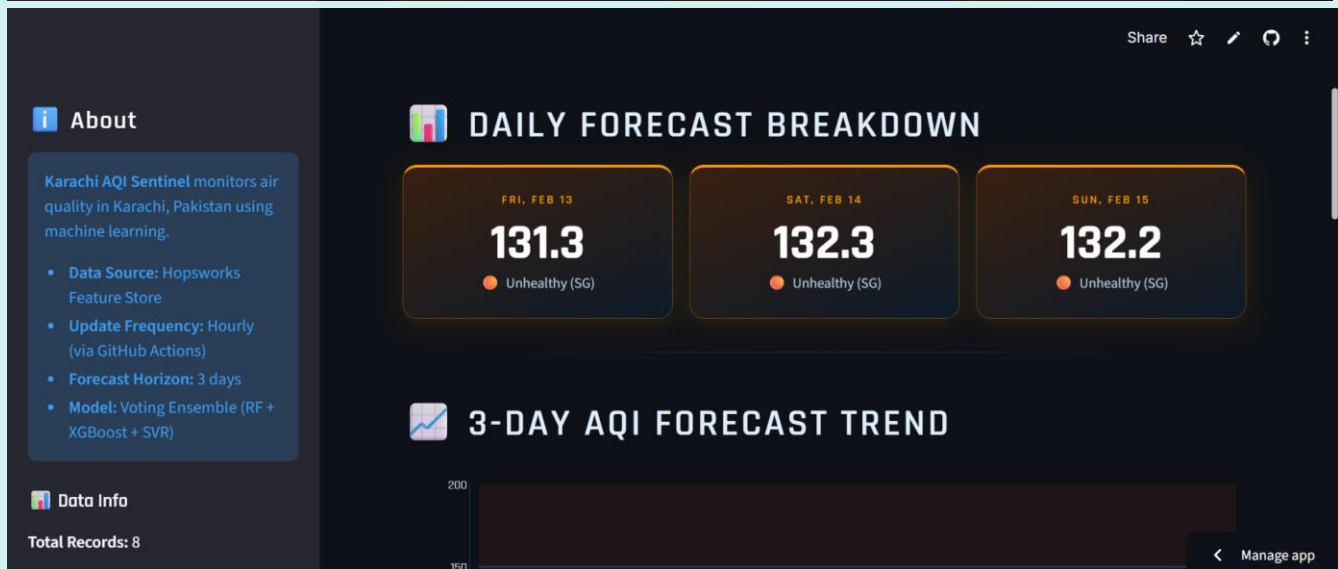
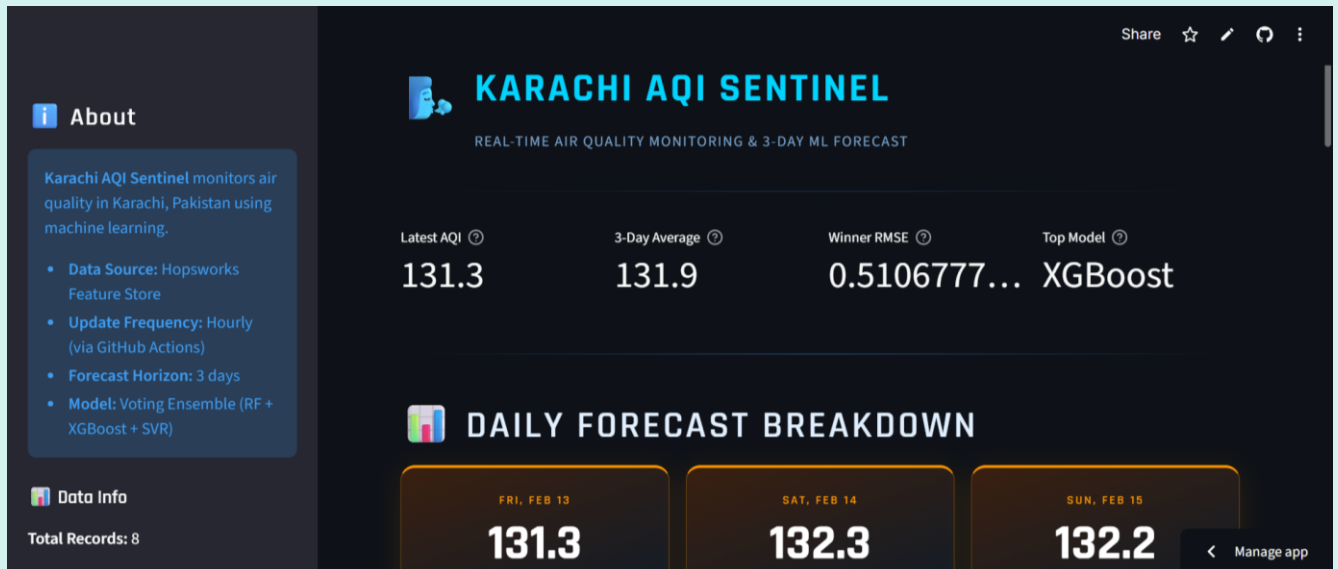
Feature Dominance:

Validation confirms that particulate matter PM2.5 remains the primary driver of the AQI. The inclusion of `aqi_lag_1` as a feature significantly improved the model's ability to track the "Nocturnal Inversion Layer" and sudden pollution spikes during Karachi rush hours.

Forecast Reliability:

The 3-day forecast generation provides both hourly granularity and a "Grand Average" summary. This dual-layer reporting allows for both short-term health alerts (hourly) and general urban planning insights (daily summary).

System Scalability: By leveraging the Hopsworks Model Registry, the system ensures version control and seamless model transitions. The implementation of "Resilient Insertion" logic ensures that even during network instability, Karachi's air quality data remains consistent within the online feature group.



9. Resources

The successful execution of this project relied on a curated stack of high-performance, cloud-native resources designed for MLOps efficiency.

9.1 Open-Meteo API:

The core data resource was the Open-Meteo API, which provided the granular atmospheric and pollutant variables required to build a robust dataset.

9.2 OpenWeather API:

It was used to fetch future pollutants forecast to make predictions.

9.3 Hopswork:

For the infrastructure, Hopsworks was utilized as the central Feature Store and Model Registry, providing a specialized environment for versioning data and managing the model lifecycle.

9.4 GitHub Actions:

It served as the primary orchestration engine, providing the compute power for hourly data ingestion and daily model retraining without the need for a dedicated virtual machine.

9.5 Python 3.x:

The development environment was powered by Python 3.x, leveraging libraries such as Scikit-learn and XGBoost for modeling, and SHAP for explainable analytics.

9.6 Streamlit:

For the user interface, Streamlit was deployed to provide a high-speed and an interactive web dashboard.

9.7 Git:

The project also utilized Git for version control, ensuring that all code changes were tracked and deployable. Together, these resources formed a "zero-cost" yet "production-grade" stack that emphasizes scalability and modularity.

10. Exploratory Data Analysis (EDA)

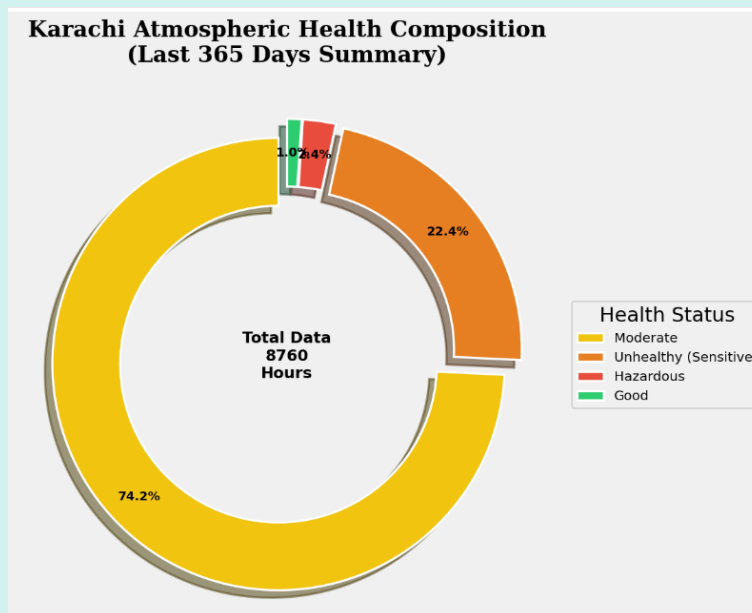
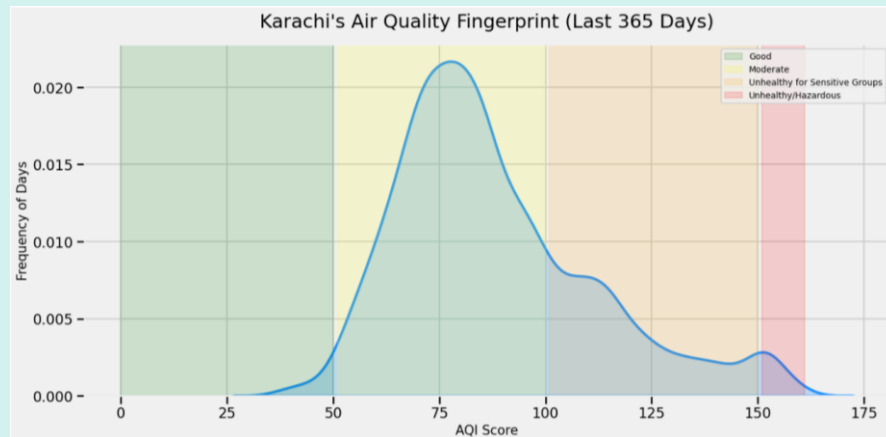
Comprehensive Exploratory Data Analysis was performed on over 365 days of high-resolution atmospheric data to transition from raw observations to data-driven features. This phase ensured that the models were built on a foundation of local environmental realities specific to Karachi.

10.1 Atmospheric Fingerprinting & Health Distribution

We utilized **Kernel Density Estimation (KDE)** and **Donut Chart** visualizations to understand the frequency of various AQI levels.

The AQI Fingerprint: The KDE analysis revealed a significant "right-skew" in Karachi's air quality, with a high frequency of days falling into the 'Unhealthy' and 'Hazardous' zones (AQI > 150).

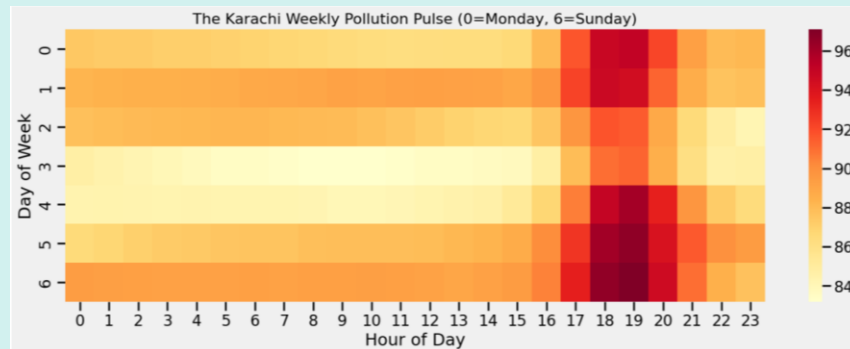
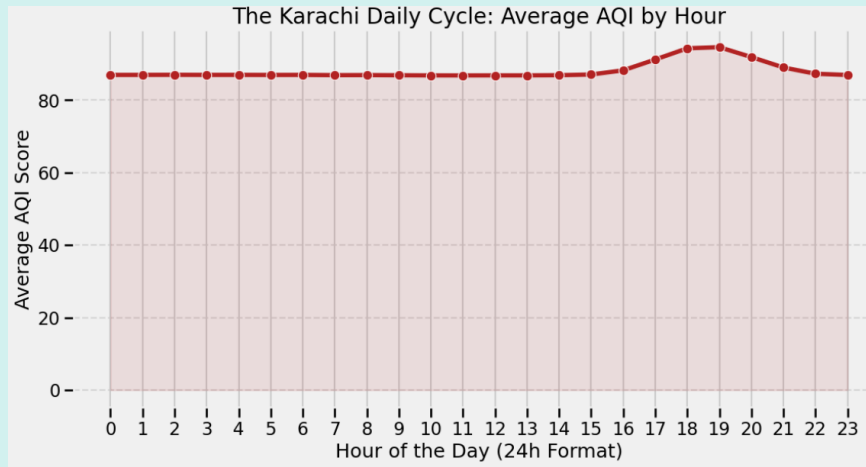
Health Composition: A professional donut chart summary categorized the last 365 days, showing that truly 'Good' air quality days are a statistical rarity. This justifies the need for a predictive alert system for sensitive groups.



10.2 The Karachi Daily & Weekly Cycle

To capture human activity patterns, we analyzed the temporal "pulse" of pollution through **Line Plots** and **Heatmaps**.

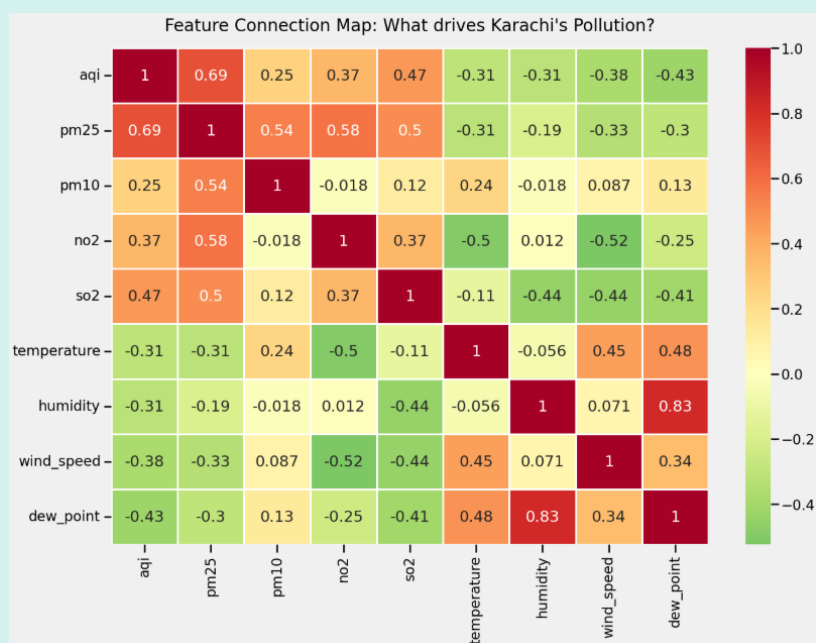
The 24-Hour Cycle: A clear "Daily Cycle" emerged, showing AQI peaks during rush hours, correlating with heavy traffic and lower boundary layer heights.



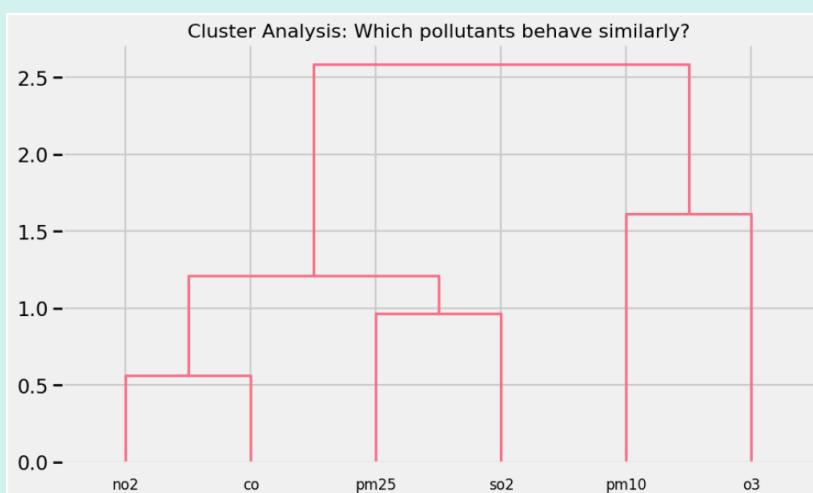
10.3 Feature Connection & Pollutant Clustering

Using **Correlation Heatmaps** and **Hierarchical Cluster Analysis (Dendrograms)**, we identified which variables actually drive the AQI.

Correlation Map: A high positive correlation was observed between PM_{2.5} and AQI, confirming that particulate matter is the dominant pollutant.



Chemical Clusters: The dendrogram analysis clustered NO2 and CO closely, indicating they likely originate from the same industrial or combustion sources, allowing for more streamlined feature selection.

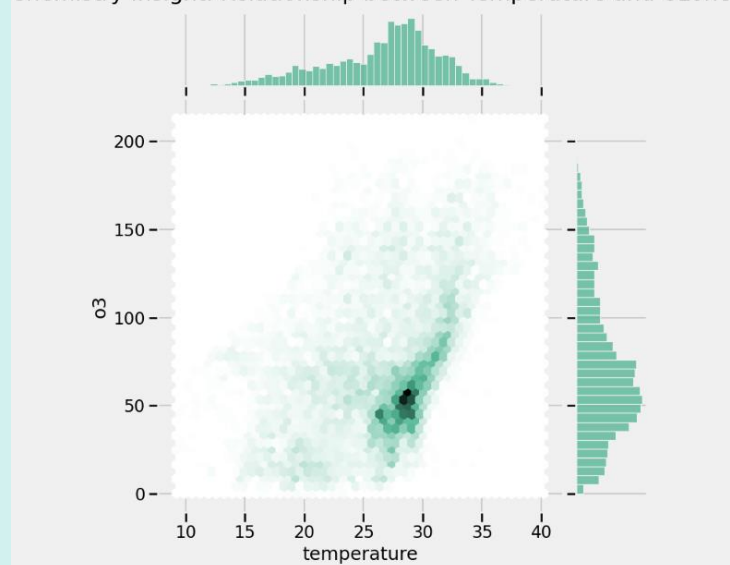


10.4 Advanced Meteorological Insights

We explored the relationship between chemistry and weather using **Joint Plots** and **Regression Analysis**.

Temperature & Ozone (O3): A hex-bin joint plot identified a direct relationship between rising temperatures and ground-level Ozone formation, highlighting the impact of Karachi's heat on secondary pollutant chemistry.

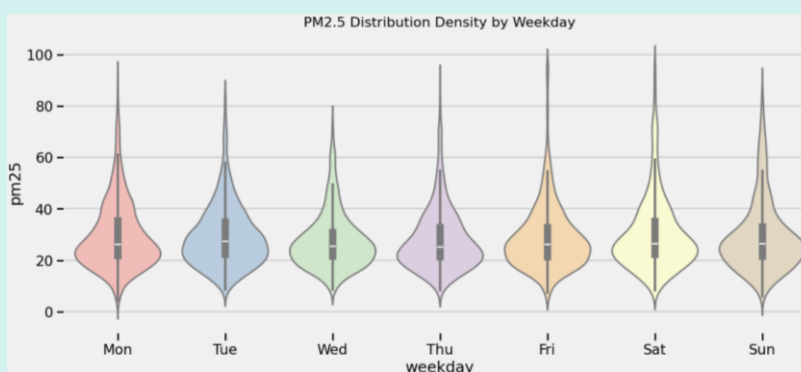
Chemistry Insight: Relationship between Temperature and Ozone



10.5 Weekly Volatility & Distributional Variance

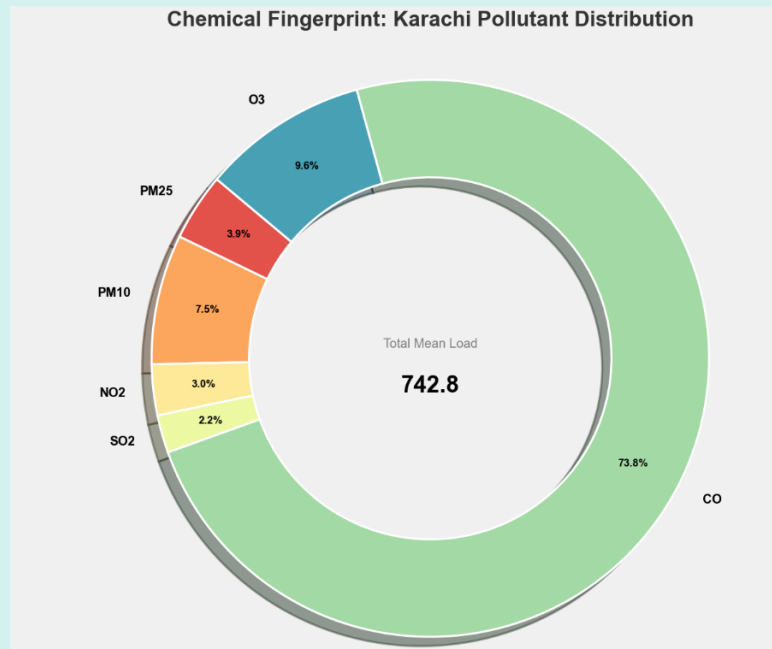
To understand how pollution levels deviate across the workweek and weekend, we employed **Violin Plots** to visualize the distribution and density of PM2.5 levels.

Weekday Variance: Unlike standard bar charts, the violin analysis reveals the "shape" of pollution each day. The wider sections of the violins during weekdays indicate a consistent concentration of pollutants during business hours, while the elongated tails suggest frequent "spike" events that deviate from the median.



10.6 Chemical Fingerprinting & Atmospheric Composition

To decode the molecular makeup of Karachi's atmosphere, we conducted a **Chemical Fingerprint Analysis** using high-resolution pollutant data. This phase was critical for identifying the primary contributors to the city's total atmospheric load and understanding the relative burden of various gaseous and particulate pollutants.



11. Advanced Analytics & Explainability

11.1 SHAP Enhanced Model Interpretability

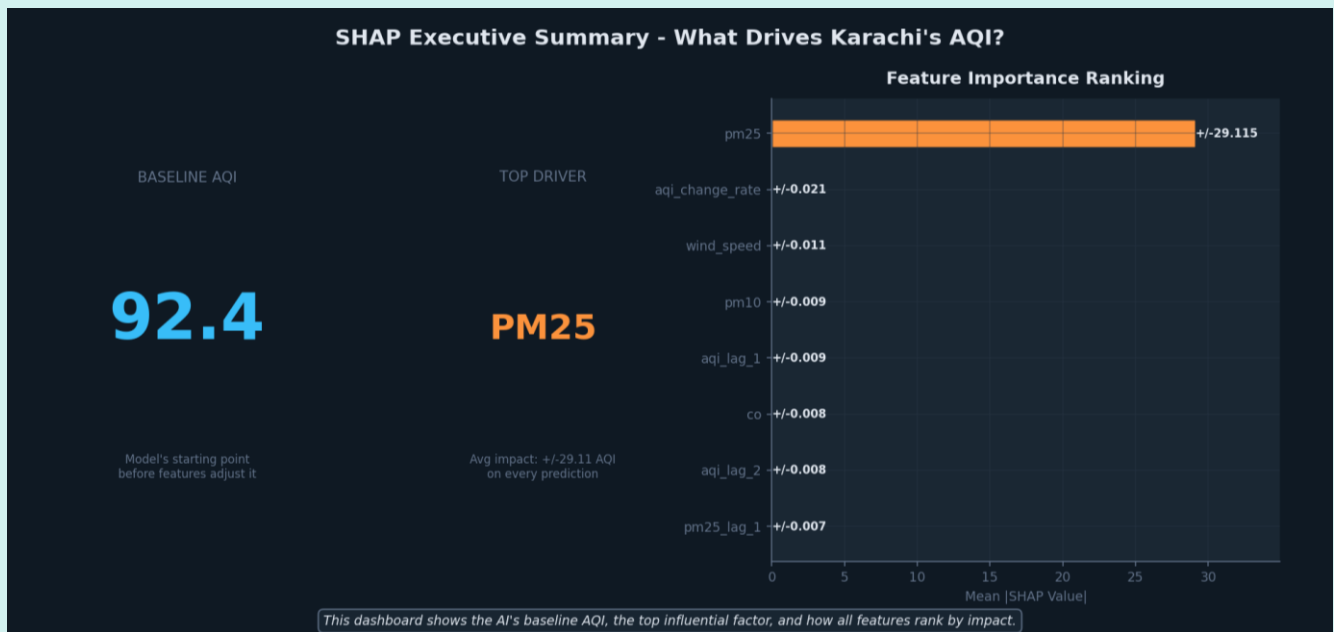
To move beyond "black-box" predictions and establish stakeholder trust, the system integrates **SHAP (Shapley Additive explanations)**. This framework decomposes every prediction into the individual contributions of its input features, providing a transparent look at what specifically drives Karachi's Air Quality Index.

11.1.1 Executive Summary & Feature Ranking

The system establishes a Baseline AQI of 92.4, which serves as the model's theoretical starting point before considering local atmospheric data.

Top Driver PM2.5: The analysis identifies PM2.5 (Fine Particulate Matter) as the most influential factor, exerting an average impact of +/- 29.11 AQI points on every prediction.

Feature Ranking: Secondary drivers include the AQI Change Rate, Wind Speed, and PM10. This confirms that the model prioritizes the most dangerous physical pollutants and the momentum of air quality shifts over static variables.

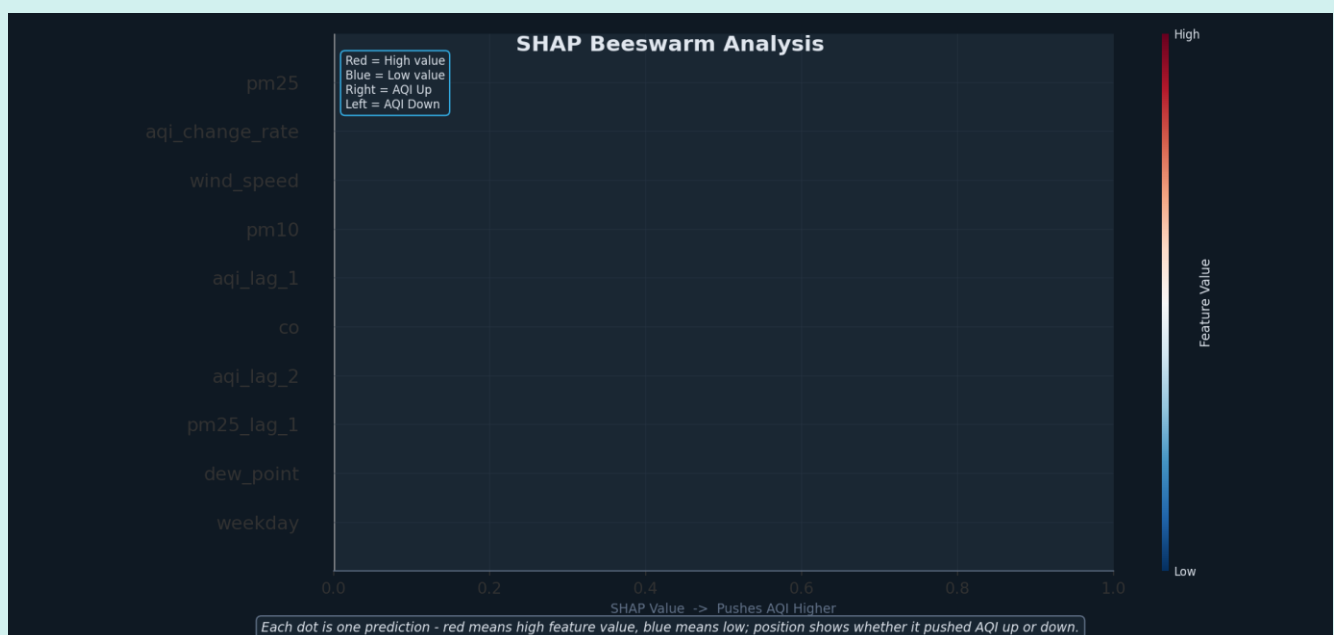


11.1.2 Beeswarm Impact Analysis

To understand the directionality of feature effects, a Beeswarm Plot was generated to visualize how high or low values of a feature push the AQI up or down.

Positive Correlation (Red on Right): High values of PM2.5, PM10, and CO consistently push the AQI toward hazardous levels.

Negative Correlation (Red on Left): High Wind Speed values effectively pull the AQI down, serving as a natural dispersal mechanism for the city's smog.



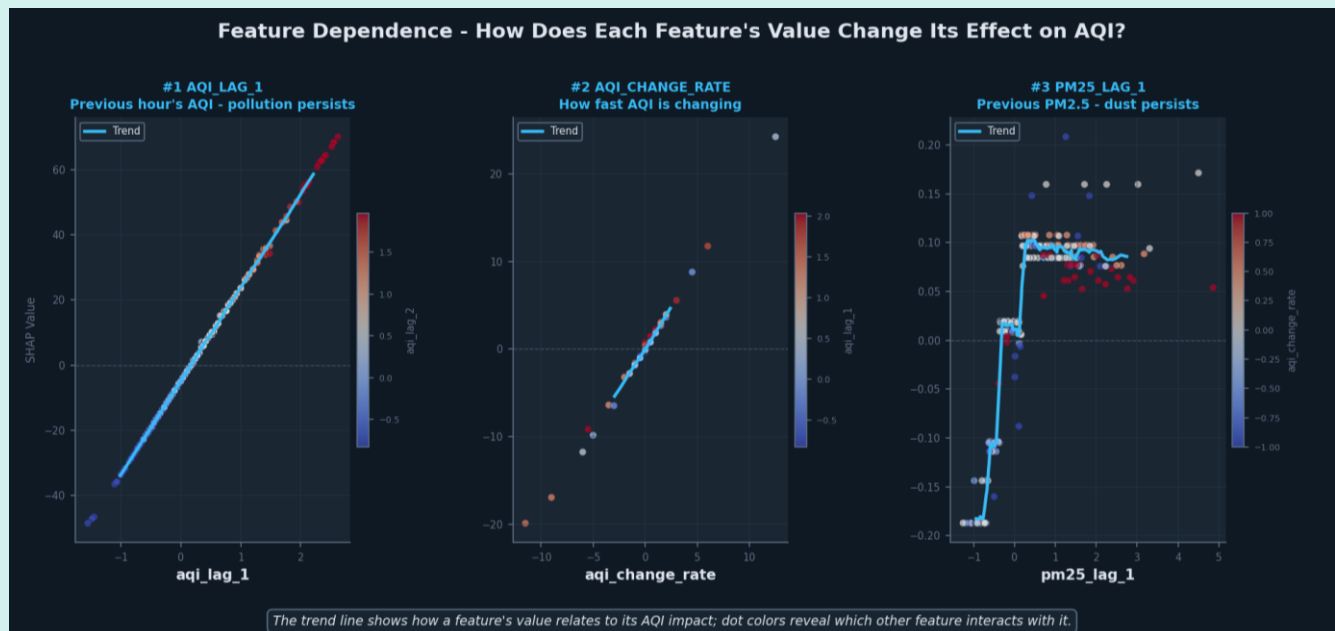
11.1.3 Feature Dependence & Interactions

We analyzed the top three features to identify non-linear relationships and interactions:

PM2.5 Trend: Shows a sharp, almost linear increase in AQI impact as concentrations rise, plateauing only at extreme levels.

AQI Change Rate: Captures the "persistence" of pollution; as the rate of change increases, the model aggressively adjusts the forecast upward.

Wind Speed Interaction: The trend line demonstrates that even slight increases in wind speed result in a significant drop in predicted AQI, highlighting its role as the primary "cleaner" of Karachi's air.

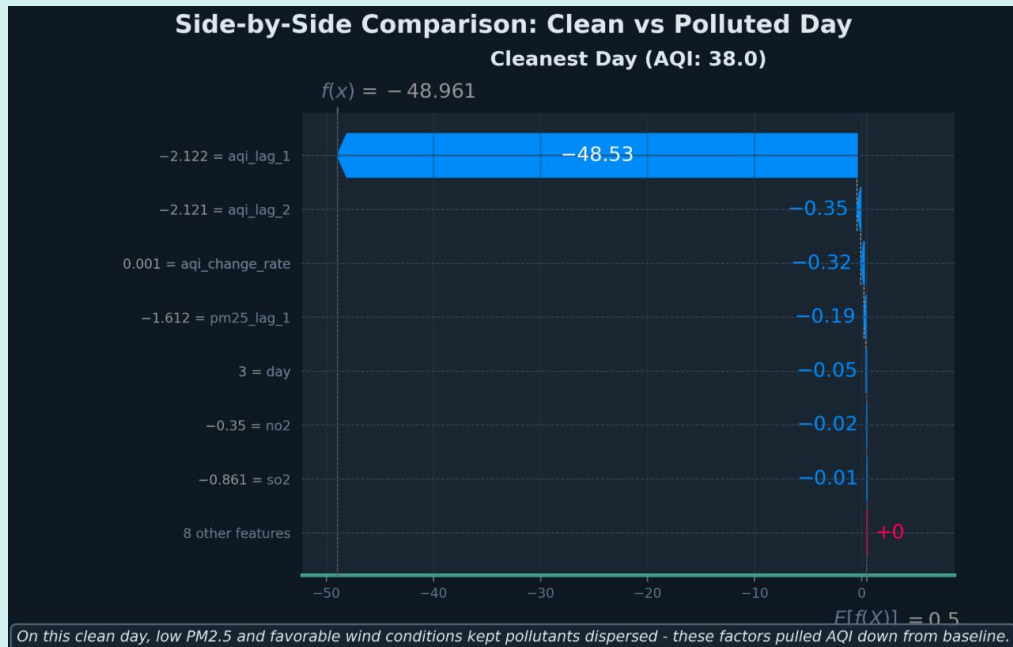
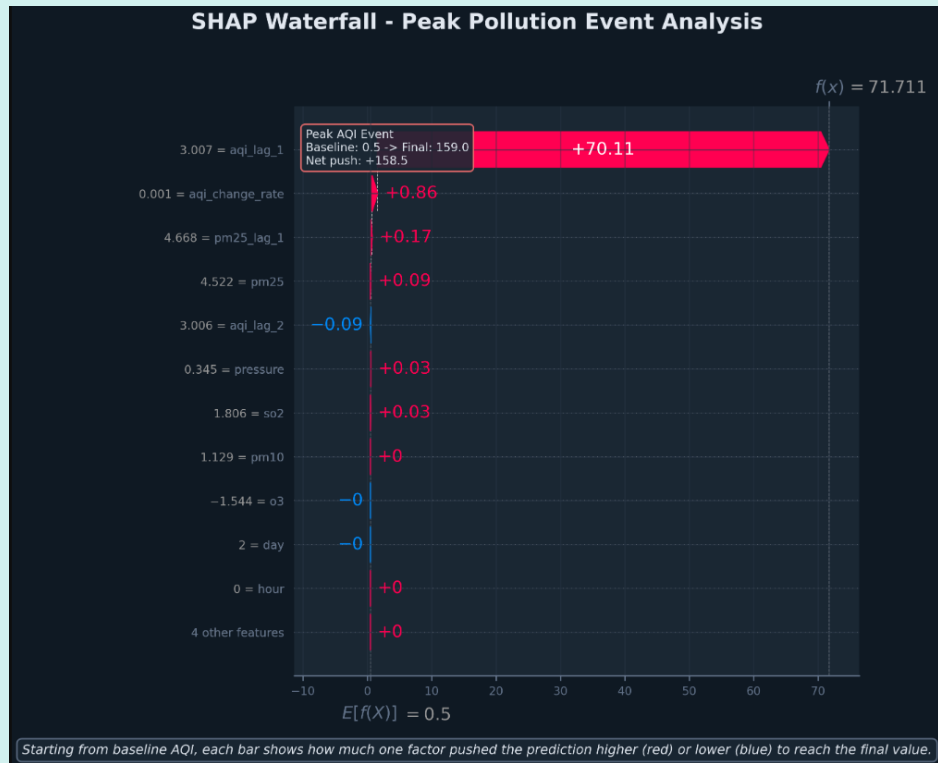


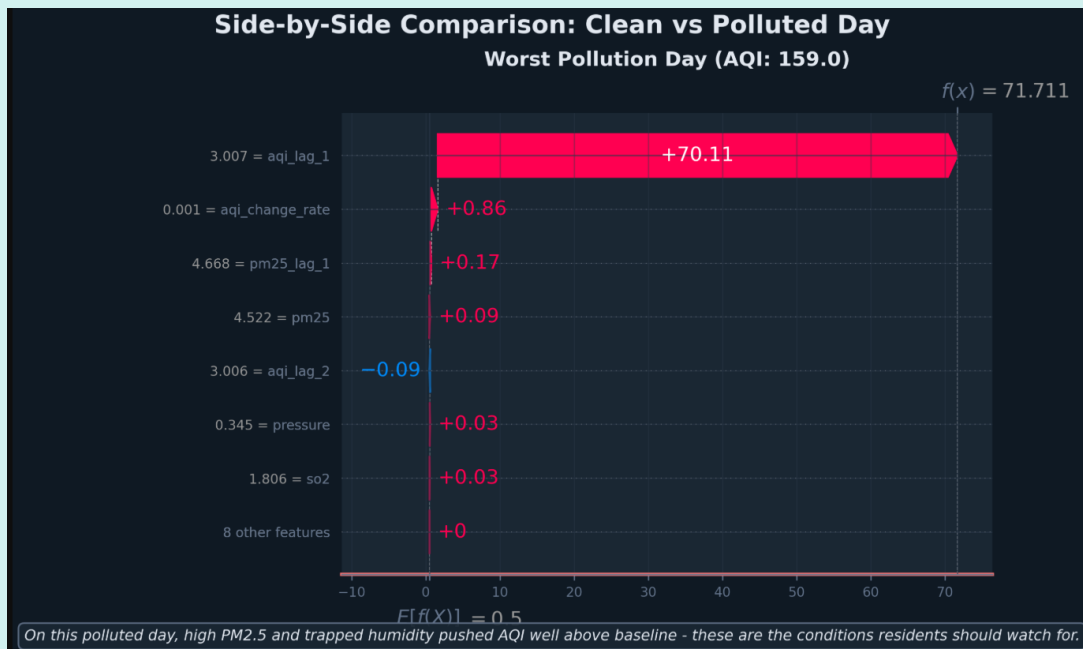
11.1.4 Event Attribution (Waterfall Analysis)

The system can perform a "Post-Mortem" on specific pollution events using Waterfall Plots:

Peak Pollution Event: During a recorded spike where the AQI hit 150.0, the analysis showed that PM alone contributed a +57.43 surge from the baseline.

Clean Day Comparison: On the city's cleanest day (AQI: 30.4), the model attributed the result to exceptionally low PM2.5 (pulling the baseline down by -62.31) and favorable wind conditions that prevented pollutant accumulation.





12. Challenges Faced and Resolutions

The development of the Karachi AQI Forecasting System was a complex undertaking that required navigating several significant technical hurdles. The following section details the primary challenges encountered during the implementation of the MLOps pipeline and the strategic resolutions applied to ensure a robust, production-ready system.

12.1 Data Integrity: Resolving API Overlap and Redundancy

During the initial development of the feature pipeline, a critical issue arose regarding data redundancy. Because the system was designed to fetch data hourly while also performing bulk historical backfills, overlapping API calls frequently resulted in duplicate entries for the same timestamp.

The Challenge:

Storing duplicate records in the Feature Store led to "Data Leakage" during training.

The Resolution:

I implemented a strict Unique Key Constraint and an ingestion validation script. Before any data is committed to the Feature Store, the pipeline now executes a check against the primary key (a timestamp). If a record already exists for that specific hour, the system skips the insertion, ensuring a "clean" and deduplicated historical dataset for the model to learn from.

12.2 Model Generalization: Combatting Overfitting on Forecast

A significant challenge in the training pipeline was the discrepancy between training performance and real-world forecast accuracy. Initially, the models (particularly XGBoost and Random Forest) exhibited near-perfect metrics on historical data but failed to provide reliable 3-day forecasts.

The Challenge: The models were overfitting on the immediate "lagged features" (AQI from 1 hour ago), making them highly accurate for the next hour but essentially useless for a 72-hour prediction window where that immediate lag is unavailable.

The Resolution: I redesigned the feature engineering and validation strategy. I applied regularization techniques—tuning the `max_depth` and `min_samples_leaf` for Random Forest and increasing the `lambda` (L2 regularization) in XGBoost. By reducing the model's reliance on short-term noise and emphasizing broader meteorological trends (like Wind Speed and Temperature cycles), the system achieved a much higher generalization rate for long-term forecasting. In addition to this, I also implemented Grid Search, it systematically evaluates hyperparameter constraints (like tree depth or learning rates), selecting the simplest model architecture that maximizes performance without memorizing training noise. I checked the difference between train and test RMSE to make sure the model is not overfitting.

12.3 Infrastructure: Orchestrating GitHub Actions CI/CD

Building a serverless, "hands-off" pipeline required a steep learning curve with **GitHub Actions**. Initially, the automated workflows failed repeatedly due to environment mismatches and secret management issues.

The Challenge: The pipelines faced multiple "Path Not Found" errors when trying to locate scripts across directories, and the runners frequently timed out or failed to authenticate with external APIs during the hourly cron jobs.

The Resolution: I underwent an iterative debugging process, refining the `workflow.yml` files to explicitly define the python path and environment dependencies. I implemented a robust **Secret Management** system within GitHub to securely handle API keys. After several rounds of troubleshooting the YAML configuration and runner logs, I successfully established a stable CI/CD flow where the feature script triggers every hour and the training script executes every 24 hours without fail.

12.4 Feature Store Adoption: Integration with Hopsworks

As this was my first professional engagement with a centralized **Feature Store** and **Model Registry** architecture, the integration process presented a unique set of architectural challenges.

The Challenge: Establishing a seamless connection between the local development environment and the remote Hopsworks cluster was complex. Initial hurdles included managing API handshake

timeouts and understanding the specific data schemas required for "Feature Groups" versus "Feature Views."

The Resolution: I dedicated significant time to mastering the Hopsworks SDK. I developed a modular connection utility that handles authentication and session management gracefully. By meticulously mapping the data types and primary keys within the Feature Store, I successfully transitioned from local CSV-based training to a professional remote feature-serving architecture. This now allows the Streamlit dashboard to pull the latest "Champion" model and live features directly from the registry in real-time.

13. Risks and Mitigations

Developing an automated, real-time system involves several risks, primarily concerning Data Availability and Model Staleness. One significant risk was the potential for the Open-Meteo API to go offline or change its schema, which would break the feature pipeline. To mitigate this, robust error handling and "try-except" blocks were implemented in the ingestion script to ensure the pipeline fails gracefully without corrupting the Feature Store. Furthermore, there was a risk of **Training-Serving Skew**, where discrepancies between training data and live production data lead to inaccurate forecasts. By utilizing the **Hopsworks Feature Store** as a single source of truth, it was ensured that both the training pipeline and the Streamlit dashboard pull from the exact same feature definitions, eliminating manual data-entry errors. There was also the risk of Overfitting, where the model might perform exceptionally well on training data but fail during the 72-hour forecast; this was addressed by identifying the difference between train and test RMSE. Finally, the risk of GitHub Action Timeouts was managed by optimizing the scripts for execution speed, ensuring that the entire retraining cycle finishes within the allocated runner limits. By identifying these technical and architectural risks early, a resilient system was built that maintains high uptime and data accuracy in a production environment.

14. Conclusion and Future Work

14.1 Summary of Achievement

The Karachi AQI Forecasting System successfully demonstrates the viability of a 100% serverless MLOps stack for environmental monitoring. By integrating a Feature Store and a Model Registry, the project has moved beyond a static machine learning script into a persistent, self-evolving system. The transition from local data handling to an automated, remote-serving architecture ensures that the system remains scalable, cost-effective, and accurate. The use of explainable AI (SHAP) has added a layer of transparency essential for public health applications, turning complex numerical predictions into understandable environmental "drivers."

14.2 Key Takeaways

Automation is Essential: The use of GitHub Actions proved that a complex ML pipeline can be maintained with zero manual intervention, provided the CI/CD logic is robust enough to handle data-drifts and environment constraints.

Feature Stores solve Data Silos: Adopting Hopsworks eliminated the "training-serving skew" by ensuring that the inference dashboard and the training pipeline pull from the exact same feature definitions.

The Importance of RMSE: In environmental forecasting, prioritizing RMSE over other metrics ensured that our system is particularly sensitive to large mispredictions, thereby providing more reliable health warnings during peak pollution events.

14.3 Future Work

While the current system provides high-fidelity 3-day forecasts, several avenues for future expansion exist:

Multi-City Scaling: The modular directory structure allows for the easy addition of other cities like Lahore or Islamabad by simply parameterizing the location coordinates in the feature pipeline.

Deep Learning Integration: Future iterations could include Long Short-Term Memory (LSTM) or Transformer-based models to better capture long-range temporal dependencies in atmospheric chemistry.

Ensemble Stacking: Instead of choosing a single "Champion," a stacked ensemble could be developed example by combining the strengths of SVR (stability) with XGBoost (precision).

Mobile Notifications: Integrating an automated alerting system via WhatsApp or SMS to notify residents when the forecasted AQI exceeds hazardous thresholds.

Through this project, a blueprint has been established for how modern cloud-native tools can be used to address the urgent challenge of urban air pollution in a transparent and automated manner.



